

Zero to Hero Study Handbook: Legal Document Summarizer & Clause Analyzer

This handbook is a static-analysis guide to the repository. Every section is grounded in the current code and artifacts in this repo.

Module 1: Foundations & Architecture

What this project does

This project builds a local legal-text analysis pipeline that converts raw legal text into a structured `LegalAnalysis` JSON object.

Primary output schema (defined in `src/legal_clause_analyzer/schemas.py`, class `LegalAnalysis`):

- `executive_summary`
- `executive_bullets`
- `clause_explanations`
- `risk_level`
- `risk_rationale`
- `liabilities`
- `obligations`
- `rights`
- `restrictions`
- `red_flags`
- `recommended_review_areas`
- `evidence_spans`

Main use cases shown by code and docs:

- Clause-level legal analysis from LexGLUE-style text.
- Baseline vs fine-tuned comparison for legal output quality.
- Strict run validation for auditable end-to-end artifacts.

Core paradigms and patterns used here

1. Typed configuration pattern
 - Implemented in `src/legal_clause_analyzer/settings.py` via Pydantic models (`AppConfig`, `ProjectConfig`, `TrainingConfig`, etc.).
 - YAML (`configs/default.yaml`) is parsed and validated before runtime.
2. Deterministic data-pipeline pattern
 - Data is loaded and normalized from LexGLUE in `src/legal_clause_analyzer/data/lexglue_loader.py`.
 - Rule-based supervision targets are generated in `src/legal_clause_analyzer/data/instruction_builder.py` + `src/legal_clause_analyzer/data/policy_mapping.py`.
3. Schema-constrained generation pattern

- Baseline prompt enforces a fixed JSON contract in `src/legal_clause_analyzer/baselines/generation` (`SYSTEM_PROMPT`, `build_prompt`).
 - Model output is parsed with fault tolerance (`try_parse_json`, `safe_analysis_from_text`).
4. Adapter-based fine-tuning pattern (QLoRA/LoRA)
 - Implemented in `src/legal_clause_analyzer/finetune/train.py` with `SFTTrainer` + PEFT (`LoraConfig`, `get_peft_model`).
 - Optional backend preference order: Unsloth, then TRL/PEFT.
 5. Reliability and fallback pattern
 - Strict mode flags (`--strict-live`) exist in multiple scripts.
 - Deterministic fallback outputs are used when strict mode is off (see `scripts/run_baselines.py` and `src/legal_clause_analyzer/inference/pipeline.py`).

Architecture and component interaction

Main orchestrator: `scripts/run_all.py`

Core modules:

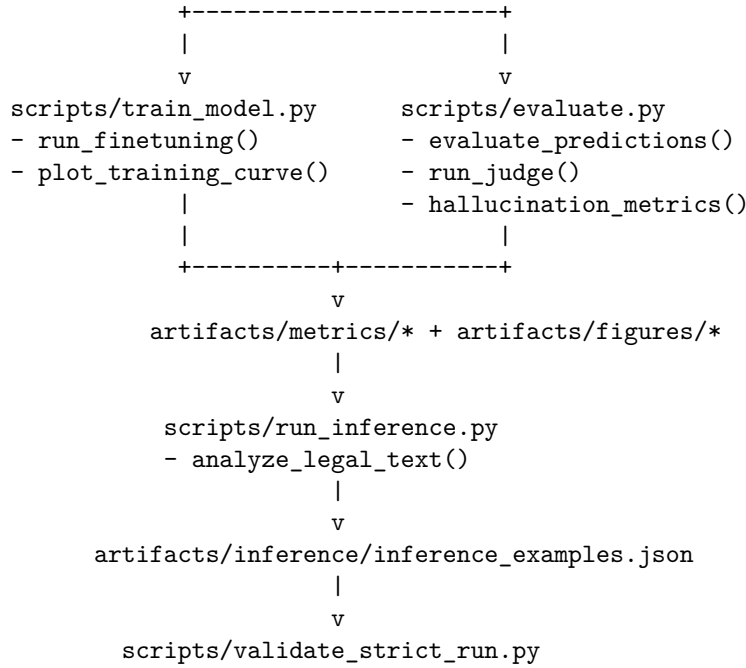
- Data prep: `scripts/prepare_data.py`
- Baselines: `scripts/run_baselines.py`
- Fine-tuning: `scripts/train_model.py`
- Evaluation: `scripts/evaluate.py`
- Inference examples: `scripts/run_inference.py`
- Strict validator: `scripts/validate_strict_run.py`

ASCII flow:

```

configs/default.yaml + configs/policy_mapping.yaml
    |
    v
  scripts/prepare_data.py
    - load_config()
    - build_dataset_registry()
    - row_to_supervision()
    |
    v
data/processed/*_supervised.jsonl + dataset_report.json
    |
    v
  scripts/run_baselines.py
    - run_baseline_generation()
    |
    v
artifacts/runs/<run_name>/predictions_*.jsonl
    |

```



Module 2: Repository Map

Focus files for first-time contributors:

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Con-figs/Variables
pyproject.toml	Project metadata, dependencies, Python version, tooling	N/A	requires-python ==3.12.10, [tool.pytest.ini_options], [tool.ruff]
configs/default.yaml	Runtime/training/evaluation parameters	N/A	project, models, runtime, sampling, training, evaluation, paths
configs/policy_map.yaml	Domain-specific label-to-legal-rule mapping	N/A	risk_levels, liability_buckets, keyword lists
src/legal_clause_analyzer/settings.py	Models + config loading	AppConfig, load_config, _ensure_dirs	RuntimeConfig.backend, TrainingConfig.trainer_backend

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Con-figs/Variables
src/legal_clause_analyzer/schemas.py	LegalAnalysis, models for outputs and rows	LegalAnalysis, EvidenceSpan, SupervisedRow	Output field names used across training/eval/inference
src/legal_clause_analyzer/data/lexical_examples.py	load/normalize/sample with split integrity	BaseExample, load_lexglue_split, build_dataset_registry	Sampling limits from config.sampling.*
src/legal_clause_analyzer/data/policies.py	infer_applicable, risk/liability/keyword extraction	infer_applicable, infer_liabilities, infer_obligations, infer_red_flags	Policy keys from configs/policy_mapping.yaml
src/legal_clause_analyzer/data/instructions.py	convert into supervised instruction-output rows	convert_to_supervised, _build_summary, _evidence_spans	Dataset-specific instruction text
scripts/prepare_dataset.py	Build processed train/val/test JSONL and dataset report	main, _distribution_plot	--config, -policy
src/legal_clause_analyzer/baselines/modified_model.py	for generation + robust JSON parsing	ModifiedModel, GenerationConfig, try_parse_json	request_timeout_seconds
src/legal_clause_analyzer/baselines/generation.py	+ parse-safe LegalAnalysis conversion	BaseGeneration, build_prompt, safe_analysis_from_text, run_baseline_generation	SYSTEM_PROMPT
scripts/run_baseline.py	Run a baseline systems on test rows	_baseline_specs, main	--strict-live, cfg.evaluation.max_eval_examples
src/legal_clause_analyzer/finetune/train.py	backend/model selection and TRL training	train, _load_trl_peft_bundle, _load_unsloth_bundle	training.*, union of config
src/legal_clause_analyzer/finetune/ollama_adapter.py	adapter and generate inference outputs	OllamaAdapter, generate_with_adapter	_load_4bit, bf16, memory limits
scripts/train_model.py	Orchestrate training + GPU guard + runtime report	main, _start_ollama_gpu_guard, _gpu_snapshot	--strict-live

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Con-figs/Variables
src/legal_clause_analyzer/evaluation/heatmap_predictions.py	Analysis/evaluation, classification, extraction metrics	summarization_metrics, classification_metrics	metrics_files/keys
src/legal_clause_analyzer/evaluation/judgegec.py	Analysis/evaluation, scoring	run_judge	JUDGE_SYSTEM_PROMPT
src/legal_clause_analyzer/evaluation/claim_metrics.py	Analysis/evaluation, claim diagnostics	claim_metrics	supported_claim_rate, missing rates
scripts/evaluate.py	Evaluate systems, run judges, generate reports/plots	main, _run_judges, _run_finetuned_predictions	--skip-finetuned, --strict-live
src/legal_clause_analyzer/inference/fallback.py	Analysis/inference, analysis API with fallback policy	analyze_legal_text, _fallback_analysis	strict_live, behavior
scripts/run_inference.py	Execute sample legal-text inference and persist outputs	main, EXAMPLES	--strict-live
scripts/validate_asserts.py	Asserts, end-to-end artifact correctness	main, _fail_if, EXPECTED_SYSTEMS	--require-notebooks
scripts/run_all.py	End-to-end pipeline entrypoint	main, _run	--strict-live, --skip-notebooks, --skip-finetuned-eval
data/processed/	Processed supervision data and dataset report	N/A	train_supervised.jsonl, validation_supervised.jsonl, test_supervised.jsonl, dataset_report.json
artifacts/runs/	Per-system prediction outputs	N/A	predictions_<system>.jsonl
artifacts/metrics/	Aggregated evaluation/judge/latency/training reports	N/A	system_metrics.json, judge_report.json, latency_report.json

Module 3: Core Execution Flows

Flow A: End-to-end pipeline orchestration (scripts/run_all.py)

Step-by-step:

1. Parse args in `main()`:
 - `--config`
 - `--skip-notebooks`
 - `--skip-finetuned-eval`
 - `--strict-live`
2. Execute scripts in fixed order via `_run()`:
 - `scripts/prepare_data.py`
 - `scripts/run_baselines.py`
 - `scripts/train_model.py`
 - `scripts/evaluate.py`
 - `scripts/run_inference.py`
 - notebook generation/execution scripts (unless skipped)
 - `scripts/validate_strict_run.py` (only when `--strict-live`)

Key behavior:

- `_run()` rewrites `python` to `sys.executable` for interpreter consistency.

Flow B: Data preparation and supervision generation (`scripts/prepare_data.py`)

Step-by-step:

1. Load and validate config:

```
cfg = load_config(args.config)
policy = load_policy(args.policy)
```

2. Build registry from LexGLUE:

- `build_dataset_registry(cfg)` in `data/lexglue_loader.py`.
- Internally uses `load_lexglue_split(...)` per dataset/split.

3. Convert each `RawExample` to `SupervisedRow`:

- `row_to_supervision(raw, policy=policy)`.
- Deterministic mapping from labels/text to `LegalAnalysis` fields.

4. Persist processed outputs:

- `data/processed/train_supervised.jsonl`
- `data/processed/validation_supervised.jsonl`
- `data/processed/test_supervised.jsonl`
- `data/processed/dataset_report.json`

Exact row shape (from `train_supervised.jsonl`):

```
{
  "row_id": "ledger_train_000000",
  "dataset": "ledger",
  "split": "train",
```

```

"instruction": "Analyze this legal clause...",
"input_text": "...",
"output": {
    "executive_summary": "...",
    "executive_bullets": [...],
    "clause_explanations": [...],
    "risk_level": "low|medium|high",
    "risk_rationale": "...",
    "liabilities": [...],
    "obligations": [...],
    "rights": [...],
    "restrictions": [...],
    "red_flags": [...],
    "recommended_review_areas": [...],
    "evidence_spans": [{"text": "...", "rationale": "..."}]
},
"risk_level": "medium"
}

```

Note: `scripts/prepare_data.py` adds a top-level `risk_level` duplicate for convenience in plotting/reporting.

Flow C: Baseline generation (`scripts/run_baselines.py`)

Step-by-step:

1. Load test rows:
 - `read_jsonl(.../test_supervised.jsonl)`
 - Slice to `cfg.evaluation.max_eval_examples`
2. Build baseline specs via `_baseline_specs(cfg)`:
 - `prompt_only`
 - `few_shot`
 - `granite_zero_shot`
 - `qwen_zero_shot`
3. For each row:
 - Call `run_baseline_generation(spec, input_text, generation_cfg)`.
 - On error:
 - strict mode on: raise runtime error.
 - strict mode off: use `_fallback_analysis(text)` deterministic output.
4. Save one JSONL per system under:
 - `artifacts/runs/<run_name>/predictions_<system>.jsonl`

Prediction row shape (from `predictions_few_shot.jsonl`):

```
{
  "row_id": "ledger_test_000000",
  "dataset": "ledger",
  "input_text": "...",
  "reference": {"...LegalAnalysis fields..."},
  "prediction": {"...LegalAnalysis fields..."},
  "system": "few_shot",
  "latency_seconds": 2.44
}
```

Flow D: Fine-tuning (scripts/train_model.py -> finetune/train.py)

Step-by-step:

1. Initialize run:
 - `seed_everything(cfg.project.seed)`
 - `_release_ollama_gpu_memory()`
 - `_start_ollama_gpu_guard()` to reduce VRAM contention.
2. Call `run_fineturning(cfg, train_jsonl, val_jsonl)`.
3. In `run_fineturning(...)`:
 - Require CUDA (`torch.cuda.is_available()`).
 - Read rows -> Pydantic validate -> format text using `format_for_sft(row)`.
 - Build training/eval datasets (`Dataset.from_dict()`).
 - Try backend/model combinations:
 - Backend order from `_backend_order(cfg)`.
 - Model order from `_candidate_models(cfg)`.
 - Build trainer with `SFTTrainer` and `SFTConfig`.
 - Train, evaluate, save adapter/tokenizer and reports.
4. Save outputs:
 - Adapter dir: `cfg.training.output_dir`
 - `training_metrics.json`
 - `training_curve.csv`
 - `training_backend_report.json`
 - Runtime report: `artifacts/metrics/training_runtime_report.json`

SFT text format shape (`format_for_sft`):

```
### Instruction
<instruction text>

### Input
<input legal text>
```



```
### Output JSON
<serialized LegalAnalysis JSON>
```

Flow E: Evaluation and judging (scripts/evaluate.py)

Step-by-step:

1. Optional fine-tuned prediction generation:
 - `_run_finetuned_predictions(...)` tries both base model candidates.
 - Generates `predictions_fine_tuned_adapter.jsonl`.
2. Load all `predictions_*.jsonl` and compute metrics:
 - `evaluate_predictions(rows)` -> nested metric families.
 - `flatten_metrics_for_table(metrics)` -> tabular DataFrame.
3. Run hallucination diagnostics:
 - `hallucination_metrics(rows_subset)` per system.
4. Run LLM judges:
 - `_run_judges(cfg, rows, strict_live=...)`
 - Judge models: `granite_judge`, `qwen_judge`.
5. Persist reports:
 - `artifacts/metrics/system_metrics.csv`
 - `artifacts/metrics/system_metrics.json`
 - `artifacts/metrics/hallucination_report.json`
 - `artifacts/metrics/judge_report.json`
 - `artifacts/metrics/latency_report.json`
 - optional `artifacts/metrics/scoreboard.csv`
6. Generate plots:
 - metric bars, system scoreboard, risk confusion matrix.

Metric report shape (`system_metrics.json`):

```
{
  "few_shot": {
    "summarization": {"rouge1": 0.0, "rouge2": 0.0, "rougeL": 0.0, "bleu": 0.0, "meteor": 0.0},
    "risk_classification": {"accuracy": 0.0, "precision": 0.0, "recall": 0.0, "f1": 0.0},
    "obligation_extraction": {"precision": 0.0, "recall": 0.0, "f1": 0.0},
    "liability_extraction": {"precision": 0.0, "recall": 0.0, "f1": 0.0},
    "red_flag_extraction": {"precision": 0.0, "recall": 0.0, "f1": 0.0}
  }
}
```

Flow F: Inference API + fallback strategy (src/legal_clause_analyzer/inference/pipeline.py)

Step-by-step for `analyze_legal_text(text, cfg, use_few_shot=False, strict_live=False)`:

1. Check adapter directory existence (`cfg.training.output_dir`).
2. Try fine-tuned path first (`load_finetuned_model + generate_with_adapter`).
3. If adapter path fails:
 - strict mode on: raise `RuntimeError`.
 - strict mode off: run baseline fallback (`run_baseline_generation`) with qwen style.
4. If fallback model call also fails and strict mode is off:
 - return deterministic `_fallback_analysis(text)`.

Return type: `InferenceResult` with fields:

- `analysis`: `LegalAnalysis`
- `model_used`: `str`
- `strategy`: `str`

Example artifact output file: `artifacts/inference/inference_examples.json`

Flow G: Strict run validation (scripts/validate_strict_run.py)

Validation checks:

1. Prediction files exist and are non-empty for:
 - `prompt_only`, `few_shot`, `granite_zero_shot`, `qwen_zero_shot`, `fine_tuned_adapter`
2. No fallback rows (summary prefix check: `Fallback summary from source snippet:`).
3. Judge report has non-empty entries for every expected system + both judges.
4. Training runtime report indicates `cuda_available=true`.
5. Inference examples use `strategy == "fine_tuned_adapter"` and non-fallback model.
6. Optional notebook execution outputs exist if `--require-notebooks` is set.

Module 4: Setup & Run Guide

1. Machine prerequisites

From code and manifests:

- Python 3.12.10 (`pyproject.toml`).

- uv package manager/workflow.
- Local Ollama CLI/server for baseline and judge models (`runtime.backend: ollama` in config).
- CUDA-capable GPU required for local training path (`run_finetuning` raises if CUDA absent).
- Optional `nvidia-smi` for GPU snapshots (script handles absence gracefully).

2. Dependency installation

```
cd /home/ahmad/AI/Github/Legal-Document-Summarizer-Clause-Analyzer
uv venv --python 3.12.10
source .venv/bin/activate
uv sync
```

3. Pull local Ollama models (as documented in README.md)

```
ollama pull qwen3-embedding:4b
ollama pull qwen3.5:4b
ollama pull granite4.1:3b
ollama pull glm-ocr
```

4. Configuration files and keys to understand first

1. configs/default.yaml

- Project metadata: `project.seed`, `project.hf_repo`, `project.run_name`.
- Model IDs: `models.generation_qwen_ollama`, `models.finetune_base_hf`, etc.
- Runtime controls: `runtime.max_generation_tokens`, `runtime.temperature`, `runtime.num_ctx`.
- Training controls: LoRA settings and memory limits under `training.*`.
- Output locations: `paths.*`.

2. configs/policy_mapping.yaml

- Deterministic legal mapping logic used during dataset preparation.

5. Environment variables (.env keys)

Static scan results for this repo:

- No required user-defined `.env` keys are referenced in source code.
- Runtime-set variables used internally:
 - `PYTHONHASHSEED` (set in `utils/seed.py`)
 - `PYTORCH_CUDA_ALLOC_CONF` (set default in `finetune/train.py`)

6. Typical command sequences

Full pipeline:

```
source .venv/bin/activate
python scripts/run_all.py --config configs/default.yaml
```

Strict live run:

```
source .venv/bin/activate
python scripts/run_all.py --config configs/default.yaml --strict-live
```

Stage-by-stage manual run:

```
python scripts/prepare_data.py --config configs/default.yaml --policy configs/policy_mapping
python scripts/run_baselines.py --config configs/default.yaml
python scripts/train_model.py --config configs/default.yaml
python scripts/evaluate.py --config configs/default.yaml
python scripts/run_inference.py --config configs/default.yaml
python scripts/validate_strict_run.py --config configs/default.yaml
```

7. Migration/seeding notes

- No database migrations exist in this repo.
- Data preparation/seeding is the dataset build step (scripts/prepare_data.py) which materializes data/processed/*.jsonl and dataset_report.json.

Module 5: Study Plan & Practice Exercises

Recommended reading order for new learners

1. Read schema and config first:
 - src/legal_clause_analyzer/schemas.py
 - src/legal_clause_analyzer/settings.py
 - configs/default.yaml
2. Understand data creation path:
 - src/legal_clause_analyzer/data/lexglue_loader.py
 - src/legal_clause_analyzer/data/policy_mapping.py
 - src/legal_clause_analyzer/data/instruction_builder.py
 - scripts/prepare_data.py
3. Learn baseline runtime path:
 - src/legal_clause_analyzer/baselines/model_clients.py
 - src/legal_clause_analyzer/baselines/generation.py
 - scripts/run_baselines.py
4. Learn training and inference internals:
 - src/legal_clause_analyzer/finetune/dataset_format.py

- `src/legal_clause_analyzer/finetune/train.py`
- `src/legal_clause_analyzer/finetune/infer_adapter.py`
- `src/legal_clause_analyzer/inference/pipeline.py`

5. Learn evaluation and reliability:

- `src/legal_clause_analyzer/evaluation/*`
- `scripts/evaluate.py`
- `scripts/validate_strict_run.py`
- `scripts/run_all.py`

6. Read tests to lock behavioral expectations:

- `tests/test_policy_mapping.py`
- `tests/test_instruction_builder.py`
- `tests/test_metrics.py`
- `tests/test_strict_inference.py`

Practice exercises

1. Trace one config value end-to-end
 - Task: Follow `runtime.max_generation_tokens` from YAML to actual model generation calls.
2. Explain deterministic risk mapping
 - Task: Using `policy_mapping.py`, explain exactly how `infer_risk_level` chooses high, medium, or low.
3. Reconstruct one supervision row
 - Task: For `dataset="ledger"`, list how `row_to_supervision` populates `instruction`, `executive_summary`, and `recommended_review_areas`.
4. Compare strict vs non-strict behavior in baselines
 - Task: In `scripts/run_baselines.py`, describe what changes when `--strict-live` is enabled.
5. Identify all fallback paths in inference
 - Task: In `inference/pipeline.py`, list every branch that can return non-fine-tuned output.
6. Explain how judge scores are produced
 - Task: Show how `scripts/evaluate.py` calls `run_judge` and aggregates judge outputs.
7. Map artifact outputs to pipeline stages
 - Task: Match each of these files to the script that creates it:
 - `data/processed/dataset_report.json`

- artifacts/runs/balanced_local_v1/predictions_few_shot.jsonl
- artifacts/metrics/system_metrics.json
- artifacts/inference/inference_examples.json

8. Interpret strict validator rules

- Task: List the exact conditions that cause `scripts/validate_strict_run.py` to fail.

Solution outlines

1. Config trace outline

- `configs/default.yaml` -> `load_config()` -> `cfg.runtime.max_generation_tokens`.
- Used in `scripts/run_baselines.py` (`GenerationConfig.max_tokens`), `scripts/evaluate.py` (fine-tuned inference), and `inference/pipeline.py`.

2. Risk mapping outline

- `infer_risk_level` lowercases labels/text.
- Checks `policy["risk_levels"]` ["high"] keywords first.
- Then checks medium keywords.
- Defaults to low.

3. Supervision row outline

- `instruction` from `_build_instruction(dataset_name)`.
- `executive_summary` from `_build_summary(row)`.
- Fixed review list from `recommended_review_areas` literal in `row_to_supervision`.

4. Strict baseline outline

- With `--strict-live`, any model exception raises `RuntimeError`.
- Without it, exception logs warning and `_fallback_analysis` output is written.

5. Inference fallback outline

- Adapter load/generate failure -> baseline fallback (`qwen_zero_shot` style).
- Baseline fallback failure (non-strict only) -> deterministic `_fallback_analysis`.
- Strict mode blocks fallback and raises.

6. Judge aggregation outline

- `_run_judges` iterates rows and judge specs (`granite_judge`, `qwen_judge`).
- Each score dict from `run_judge` becomes a `DataFrame`.
- Mean per score key is stored in `judge_report.json`.

7. Artifact mapping outline

- `scripts/prepare_data.py` -> `dataset_report.json`.
- `scripts/run_baselines.py` -> `predictions_*.jsonl`.
- `scripts/evaluate.py` -> `system_metrics.json`.

- `scripts/run_inference.py -> inference_examples.json`.

8. Strict validator outline

- Missing/empty predictions for expected systems.
- Any fallback summary rows in predictions.
- Missing/empty judge metrics per system+judge.
- `cuda_available=false` in training runtime report.
- Inference rows not using `fine_tuned_adapter` or using deterministic fallback.
- Missing executed notebooks when `--require-notebooks` is enabled.

Learner Verification Checklist

Use this checklist after finishing the handbook:

- Can you explain `LegalAnalysis` fields and where they are validated/used?
- Can you trace `scripts/run_all.py` and describe each stage's input and output files?
- Can you explain how `row_to_supervision` transforms a `RawExample` into a training row?
- Can you explain the difference between baseline generation fallback and strict-live behavior?
- Can you describe how fine-tuning backend/model fallback is implemented in `run_finetuning`?
- Can you explain how summarization/classification/extraction metrics are computed in `evaluation/metrics.py`?
- Can you explain how judge scoring and hallucination diagnostics complement standard metrics?
- Can you explain every strict validation rule in `scripts/validate_strict_run.py`?
- Can you identify the exact files to inspect when inference quality degrades (`artifacts/runs`, `artifacts/metrics`, `artifacts/inference`)?